



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche, Informatiche e Matematiche

Corso di Laurea Magistrale in Informatica (LM-18)

Implementazione di una pipeline per la generazione di mesh 3D a partire da modelli CAD 2D di planimetrie architettoniche

Relatore:

Prof. Nicola Capodieci

Correlatore:

Prof. Fabio Pellacini

Candidato:

Simone Vaiasinni

Matricola 201711

ANNO ACCADEMICO 2025/2026

Abstract

Floorplan3D: implementazione di una pipeline per la generazione di mesh 3D a partire da modelli CAD 2D di planimetrie architettoniche

L'abstract è il riassunto della tesi. Deve dare in poche righe (in genere 150-300 parole, una mezza pagina), una visione completa e autosufficiente del lavoro: problema, approccio, risultati, senza bisogno di leggere il resto.

1. Contesto e problema (2-3 righe) Qual è il problema generale? Perché è rilevante? Nel tuo caso: la difficoltà di ottenere modelli 3D di ambienti architettonici a partire da planimetrie 2D CAD, utili per rendering, visualizzazione, ecc.

2. La sfida specifica (1-2 righe) Cosa rende il problema difficile? Nel tuo caso: l'assenza di uno standard unico tra i modelli CAD, che rende difficile generalizzare un algoritmo.

3. Cosa hai fatto (il cuore dell'abstract, 4-6 righe) Qui descrivi Floorplan3D: un sistema/pipeline che, a partire da un file DXF con muri, porte e finestre, produce una mesh 3D esportata in GLTF. Vale la pena accennare, senza entrare troppo nei dettagli implementativi, alle fasi principali della pipeline (parsing, ricostruzione geometrica dei varchi, estrazione del grafo planare e delle facce, triangolazione ed estrusione), perché è proprio questo il contributo tecnico del lavoro.

4. Risultati/validazione (1-2 righe) Cosa hai ottenuto? Hai testato su più planimetrie reali? Hai fatto un confronto qualitativo, dei rendering finali in Blender, delle metriche sui tempi di elaborazione, sulla qualità della mesh? Anche un risultato qualitativo ("il sistema produce mesh corrette e prive di artefatti su planimetrie reali di complessità variabile") va benissimo se non hai metriche quantitative.

Indice

Abstract	ii
1 Introduzione	1
1.1 Il dominio del problema	1
1.2 I problemi dei modelli CAD	1
1.3 Obiettivi della tesi	2
1.4 Struttura della tesi	3
2 Background teorico	4
2.1 Strutture dati spaziali	4
2.1.1 Approcci alla ricerca di prossimità	4
2.1.2 L'hash spaziale	5
2.1.3 Complessità e scelta della dimensione della cella	5
2.2 Clustering di punti	6
2.2.1 L'algoritmo DBSCAN	7
2.2.2 Scelta dei parametri	10
2.3 Rappresentazione di una mesh	10
2.3.1 Topologia della mesh	11
2.3.2 Indexed Mesh	11
2.3.3 Grafi planari e Planar Straight-Line Graph	12
2.3.4 Strutture dati per la connettività	13
2.3.5 Attributi dei vertici	14
2.4 Algoritmi di triangolazione	15
2.4.1 Ear clipping	16
2.4.2 Triangolazione di Delaunay	17
3 Risultati e valutazione	19

4 Conclusioni	20
Ringraziamenti	21
Riferimenti	22

Listings

1. Introduzione

L'obiettivo del presente lavoro di tesi è quello di sviluppare un sistema in grado di generare automaticamente una mesh 3D partendo da un modello CAD 2D di una planimetria di una singola stanza o di un appartamento. Per ogni modello CAD 2D che verrà letto in input dal sistema il risultato sarà una mesh 3D con vertici, indici, normali e coordinate di texture che possono essere immediatamente utilizzate per il rendering.

1.1 Il dominio del problema

Il dominio del problema comprende i piani architettonici CAD 2D in formato DXF (Drawing Exchange Format). Mentre DWG e DXF sono standard de facto nel campo del CAD, sono diversi tra loro per quanto riguarda l'approccio alla codifica: DWG è un formato binario proprietario, più efficiente nello spazio ma meno leggibile in quanto può essere letto da poche applicazioni, mentre DXF è un formato basato sul testo (in ASCII) sviluppato in modo tale da non presentare problemi di interoperabilità con i software di terzi.

Per poter iniziare a lavorare sulla progettazione della pipeline di elaborazione è stato prima necessario raccogliere un dataset composto da modelli CAD 2D rappresentativi del dominio di applicazione: più di dieci modelli CAD sono stati raccolti da diversi siti web come `draftsperson.net`, `pincad.com` e `archweb.com`. Questi modelli erano inizialmente disponibili nel formato DWG e sono stati quindi convertiti in formato DXF e controllati con un software di editing come LibreCAD. Tra i modelli disponibili ne sono stati selezionati alcuni come studi di caso, utilizzati nel corso della tesi per illustrare le diverse fasi della pipeline.

1.2 I problemi dei modelli CAD

Il formato DXF non fornisce direttamente poligoni, bensì un insieme di entità geometriche disconnesse tra loro, quali POLYLINE, LINE, ARC ecc., organizzate in layer e blocchi. È qui che emerge la prima criticità: non esiste alcuno standard che imponga una convenzione univoca sui nomi dei layer

o dei blocchi. Un architetto può decidere di nominare i layer in un certo modo, e un altro architetto in un modo diverso. Questo vale anche per i blocchi, che molto spesso presentano nomi privi di significato.

Un secondo problema riguarda la rappresentazione geometrica degli elementi. Per quanto riguarda i muri, ad esempio, un architetto può rappresentarle con un semplice contorno privo di spessore, mentre un altro le disegna con uno spessore definito. Alcuni utilizzano polilinee che definiscono contorni già chiusi, altri invece semplici segmenti che spesso si intersecano impropriamente tra loro e risultano aperti, facendo apparire la geometria come "rotta".

Non solo i muri, ma anche le finestre presentano una notevole variabilità: alcuni architetti le descrivono come blocchi complessi composti da centinaia di segmenti, altri come semplici segmenti che collegano due pareti, altri ancora come rettangoli definiti da quattro lati, con ulteriori segmenti interni per rappresentare il vetro.

Le porte, invece, tendono a essere rappresentate in modo più uniforme tra i vari modelli, tipicamente tramite archi; in alcuni modelli, tuttavia, in particolare per le porte d'ingresso, possono essere rappresentate tramite polilinee o segmenti anziché tramite un arco.

Questa eterogeneità è la ragione principale per cui non è possibile progettare un algoritmo generale in grado di elaborare correttamente qualsiasi modello CAD. È necessaria, a monte, una fase di preprocessing volta a rendere il modello quanto più consistente possibile (Sezione ?? del Capitolo ??).

1.3 Obiettivi della tesi

Il presente lavoro si pone i seguenti obiettivi:

- progettare un parser in grado di interpretare in modo robusto le entità geometriche eterogenee presenti in un file DXF (segmenti, polilinee, archi, blocchi), riconducendole a un insieme coerente di muri, porte e finestre;
- ricostruire la topologia della planimetria a partire da primitive geometriche, unificando i vertici duplicati e ricostruendo correttamente i varchi sui muri;

- generare una mesh 3D completa di normali e coordinate texture, distinguendo pavimento, soffitto, muri, porte e finestre;
- esportare il modello in formato GLTF pronto per essere importato in Blender, così da effettuare un rendering fotorealistico della scena.

1.4 Struttura della tesi

Il resto della tesi è organizzato come segue.

Il Capitolo 2 introduce le nozioni di geometria computazionale necessarie per comprendere le scelte progettuali descritte nei capitoli successivi: strutture dati spaziali, algoritmi di clustering di punti, grafi planari, tecniche di triangolazione di poligoni e rappresentazione di mesh 3D (in particolare normali e coordinate texture).

Il Capitolo ?? descrive la progettazione e l'implementazione della pipeline, dalle assunzioni progettuali fino a ciascuna delle fasi che la compongono: preprocessing, parsing, collasso dei vertici, ricostruzione dei varchi, estrazione e classificazione delle facce, triangolazione ed estrusione, esportazione.

Il Capitolo ?? conclude il lavoro descrivendo la fase di rendering: generazione della scena in Blender e setup dei materiali, delle luci, della camera e del motore Cycles.

2. Background teorico

2.1 Strutture dati spaziali

Molti algoritmi di elaborazione geometrica richiedono, come operazione di base, di rispondere in modo efficiente a interrogazioni di prossimità tra punti: dato un punto p , trovare tutti i punti p' che distano da esso meno di una soglia r fissata. Questo problema è noto in letteratura come *fixed-radius near-neighbor problem* e può essere formalizzato come segue:

$$\text{dato } p, \text{ trovare } \{p' : d(p, p') < r\}$$

dove d è la distanza euclidea. Un approccio diretto (*naive*) consiste nel confrontare ogni punto con tutti gli altri, con una complessità $O(N)$ per ogni singola interrogazione e quindi $O(N^2)$ per elaborare un intero insieme di N punti: un costo che diventa rapidamente insostenibile al crescere della dimensione dell'insieme. Per questo motivo, quando il numero di interrogazioni è elevato, è preferibile appoggiarsi a una struttura dati pensata per organizzare i punti nello spazio e restringere la ricerca ai soli candidati plausibili.

2.1.1 Approcci alla ricerca di prossimità

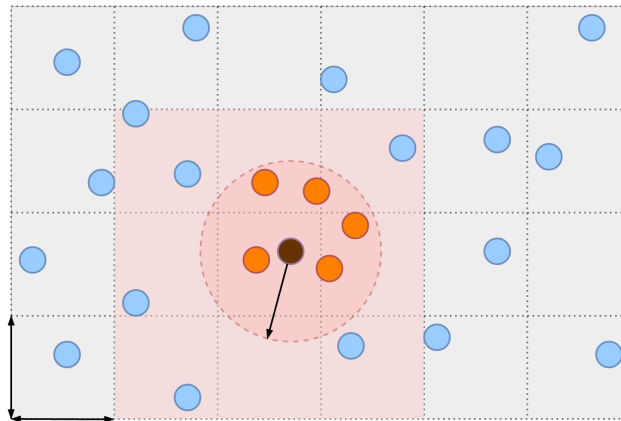
Le strutture dati più diffuse per questo tipo di problema si dividono in due grandi famiglie. La prima è quella delle strutture gerarchiche di partizionamento dello spazio, come il *K-D Tree* o il *quadtree*, che suddividono ricorsivamente lo spazio in regioni via via più piccole organizzate in un albero. Queste strutture garantiscono interrogazioni di vicinanza in tempo $O(\log N)$.

La seconda famiglia è quella delle strutture basate su una griglia regolare, tra cui l'*hash spaziale* (*spatial hash*). A differenza delle strutture ad albero, un hash spaziale non organizza i punti in una gerarchia, ma li distribuisce in *bucket* (celle) di una griglia uniforme, usando le coordinate del punto stesso come chiave di hashing. Questo approccio è più semplice da implementare, supporta inserimenti e rimozioni in tempo pressoché costante, ed è particolarmente indicato quando il raggio di ricerca r è noto a priori [1].

2.1.2 L'hash spaziale

Il funzionamento di un hash spaziale si basa su un'idea semplice: lo spazio 2D viene suddiviso in una griglia regolare, in cui ogni cella ha una dimensione fissa. A ciascun punto viene associata esattamente una cella, calcolata a partire dalle sue coordinate. L'insieme dei punti che ricadono nella stessa cella viene poi memorizzato in una struttura hash (ad esempio una `unordered_map` o una `unordered_multimap`), la cui chiave è la coppia di indici che identifica la cella.

La proprietà che rende questa struttura efficiente per le interrogazioni di prossimità è la seguente: se la dimensione della cella è scelta pari al raggio di ricerca r , due punti che distano meno di r si trovano necessariamente nella stessa cella oppure in una delle celle immediatamente adiacenti. Questo significa che, per rispondere a un'interrogazione centrata in un punto p , non è necessario esaminare l'intero insieme di punti, ma è sufficiente calcolare la cella di appartenenza di p ed esaminare unicamente le celle vicine (una griglia 3×3), effettuando un confronto esplicito di distanza solo con i punti che vi ricadono.



2.1.3 Complessità e scelta della dimensione della cella

L'inserimento di un nuovo punto ha complessità $O(1)$. Analogamente, un'interrogazione di prossimità ha complessità $O(k)$, dove k è il numero medio di punti contenuti nelle celle esaminate, indipendente dalla dimensione totale N . Nel caso peggiore, in cui i punti si concentrano in modo fortemente non uniforme (ad esempio tutti nella stessa cella), la complessità può degradare fino a $O(N)$; tale scenario è raro nella pratica.

La dimensione della cella è un parametro cruciale per le prestazioni della struttura. Se le celle sono più grandi del raggio r , il numero di celle da esaminare resta uguale ma cresce il numero di punti presenti in ciascuna cella, aumentando i confronti da effettuare. Se, al contrario, le celle sono più piccole di r , il numero di punti per cella si riduce, ma bisogna estendere la ricerca a un intorno più ampio (ad esempio una griglia 5×5). La scelta di dimensionare la cella pari a r rappresenta, in generale, il miglior compromesso tra il numero di celle da visitare e il numero di punti da confrontare all'interno di ciascuna.

Questa stessa tecnica è nota in letteratura anche con il nome di *Cell Lists*, e le sue proprietà teoriche sono state analizzate in dettaglio da Tollander de Balsch [2], che ne formalizza la complessità considerando non la singola interrogazione ma l'elaborazione dell'intero insieme di N punti. Nell'ipotesi che il numero di punti per cella sia limitato da una costante indipendente da N (i punti non si accumulano in una stessa posizione), il numero totale di confronti necessari a individuare tutte le coppie di punti vicine cresce linearmente, $O(N)$, contro $O(N^2)$ dell'approccio brute force; nel caso peggiore, in cui tutti i punti ricadono in un'unica cella, le due complessità coincidono.

Questa struttura dati verrà ripresa e applicata concretamente nella Sezione ??, nell'ambito dell'operazione di collasso dei vertici (*vertex snapping*) durante l'elaborazione della planimetria CAD.

2.2 Clustering di punti

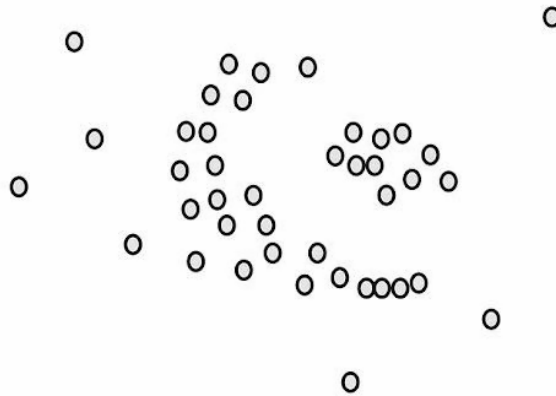
Un altro problema abbastanza comune nell'elaborazione di dati geometrici è quello di raggruppare un insieme di punti in sottoinsiemi (*cluster*) coerenti, in base alla loro posizione. A differenza della ricerca di prossimità, che risponde a interrogazioni puntuali, il clustering spaziale ha l'obiettivo di partizionare l'intero insieme di dati, individuando automaticamente quali punti appartengono allo stesso oggetto e quali, invece, sono isolati o rappresentano rumore.

La categoria degli algoritmi più adatta è quella degli *density-based*, in particolare **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*), introdotto da Ester et al. [4]. A differenza di algoritmi come *k-means*, che richiedono di specificare a priori il numero di cluster e tendono a produrre cluster di forma convessa, DBSCAN definisce i cluster come regioni di spazio

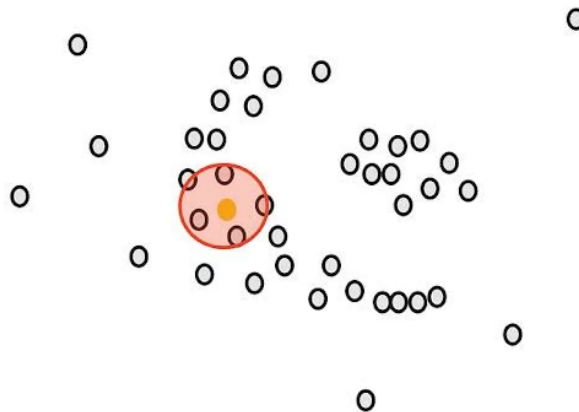
ad alta densità di punti separate da regioni a bassa densità, ed è quindi in grado di identificare cluster di forma arbitraria e di isolare automaticamente i punti isolati come rumore [8].

2.2.1 L'algoritmo DBSCAN

Prendo come riferimento l'articolo di Okan Yenigün [8]. Per capire come funziona l'algoritmo DBSCAN consideriamo il seguente esempio:

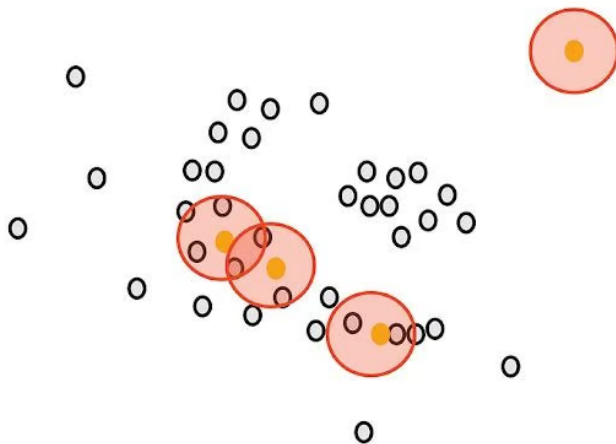


L'obiettivo è di raggruppare questi punti in gruppi (cluster). Prendiamo inizialmente un punto da cui partire e disegniamo un cerchio attorno:

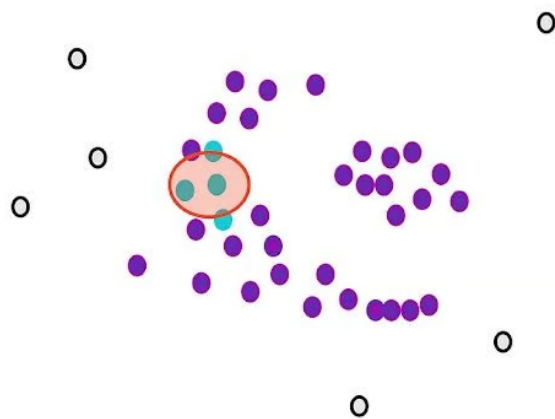


Il raggio ϵ del cerchio è il primo parametro da determinare. Dopodiché contiamo quanti punti si trovano all'interno del cerchio e vediamo che ci sono esattamente 5 punti di prossimità.

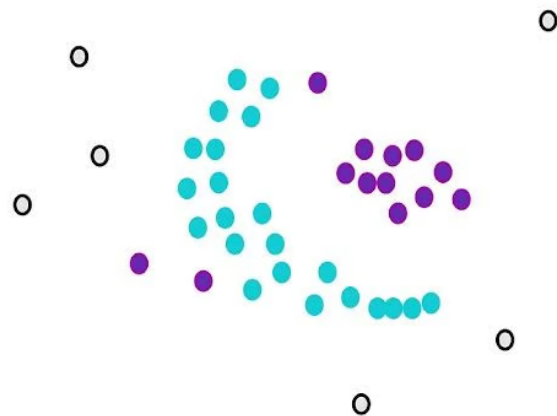
Adesso dobbiamo determinare un altro parametro, ovvero il numero minimo m di punti. Un punto si dice essere un "core point" se è vicino almeno a m punti. Se consideriamo $m = 3$, allora i seguenti punti evidenziati in viola sono core point, mentre il punto in giallo no.



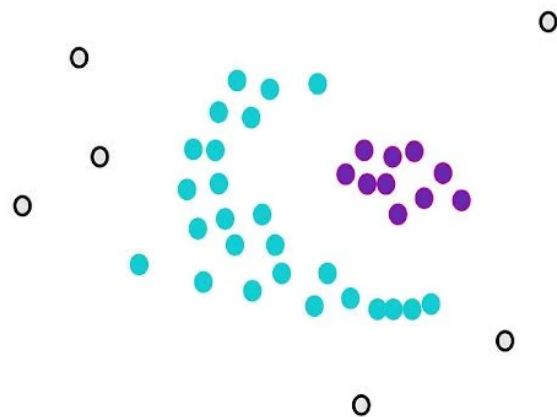
Dopo, selezioniamo a caso un core point e lo assegniamo come primo punto nel nostro primo cluster. I punti che si trovano vicino al core point vengono considerati dentro al cluster.



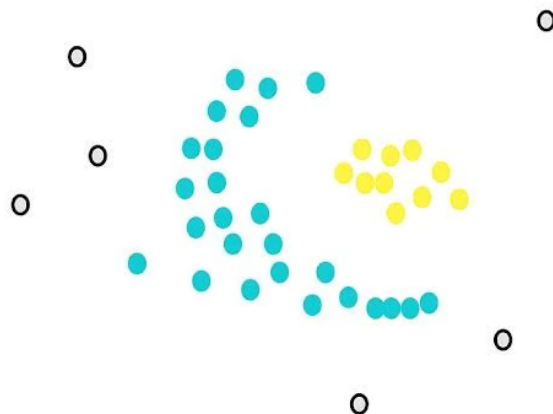
Ripetiamo il procedimento anche per gli altri punti vicini e ci fermiamo quando non possiamo più assegnare più assegnare core points al primo cluster.



Ci sono dei punti che sono vicini al primo cluster che però non sono dei core points. Disegniamo i cerchi anche su questi punti e vediamo se rientrano nel cluster. Questi punti sono i punti di frontiera (o anomalie).



Sono rimasti altri core points che non sono ancora stati assegnati. Ne selezioniamo uno a caso e ripetiamo il procedimento.



DBSCAN è un algoritmo sequenziale, è quindi importante notare che i punti non core verranno assegnati al primo cluster che soddisfa il requisito di vicinanza.

2.2.2 Scelta dei parametri

La qualità del risultato prodotto da DBSCAN dipende dalla scelta dei due parametri ϵ e m . Un valore di ϵ troppo grande tende a far collapsare cluster distinti in un unico cluster; al contrario, un valore troppo piccolo produce una frammentazione eccessiva. Analogamente, m regola la densità minima richiesta per considerare una regione un cluster: valori più alti producono cluster più densi e robusti al rumore, ma rischiano di scartare cluster legittimi composti da pochi punti.

2.3 Rappresentazione di una mesh

La maggior parte dei modelli 3D utilizzati in applicazioni reali è composta da complessi di triangoli con vertici condivisi, comunemente detti *mesh triangolari*; gestire questa struttura in modo efficiente è determinante per le prestazioni di un programma grafico, sia in termini di occupazione di memoria su disco che a runtime [6]. Una mesh triangolare non è un insieme di triangoli tra loro indipendenti, ma una rete di triangoli connessi tramite vertici e archi condivisi a formare un'unica superficie continua: è proprio questa connettività condivisa, e non la semplice somma dei singoli triangoli, a permettere di rappresentare ed elaborare la mesh in modo più efficiente rispetto a una collezione di triangoli scorrelati.

L'informazione minima necessaria per definire una mesh triangolare è l'insieme dei triangoli, ciascuno espresso come tripla di vertici, e la posizione nello spazio di ciascun vertice. La quasi totalità delle applicazioni pratiche, tuttavia, richiede di associare ai vertici (o, a seconda dei casi, agli archi o alle facce) dati aggiuntivi necessari per operazioni come il texture mapping, lo shading o l'animazione. I dati associati ai vertici sono i più comuni: a ciascun vertice possono essere associati parametri di materiale, coordinate texture, o qualunque altra grandezza il cui valore vari con continuità lungo la superficie; tali valori vengono poi interpolati linearmente all'interno di ciascun triangolo, ottenendo così una funzione continua definita sull'intera superficie della mesh.

2.3.1 Topologia della mesh

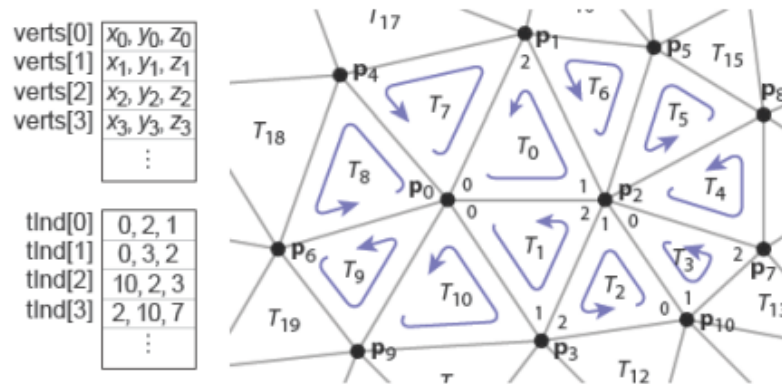
L'idea che una mesh rappresenti una superficie, e non un insieme arbitrario di triangoli, può essere formalizzata attraverso vincoli sulla sua topologia, ovvero sul modo in cui i triangoli sono connessi tra loro, indipendentemente dalla posizione dei vertici. Il vincolo più semplice e più restrittivo è che la mesh sia una varietà (*manifold*): una mesh manifold non presenta buchi o giunzioni ambigue, e separa in modo netto lo spazio interno alla superficie da quello esterno [6]. Questa proprietà è alla base del funzionamento di molti algoritmi di elaborazione geometrica, tra cui la struttura dati Half-Edge discussa nella Sezione 2.3.4, che assume implicitamente che la mesh su cui opera sia manifold.

In molte applicazioni è inoltre necessario poter distinguere il lato "esterno" di una superficie da quello "interno", proprietà nota come orientazione. Per un singolo triangolo, l'orientazione è convenzionalmente definita in base all'ordine con cui i suoi vertici sono elencati: il lato anteriore del triangolo è quello da cui i tre vertici appaiono disposti in senso antiorario.

2.3.2 Indexed Mesh

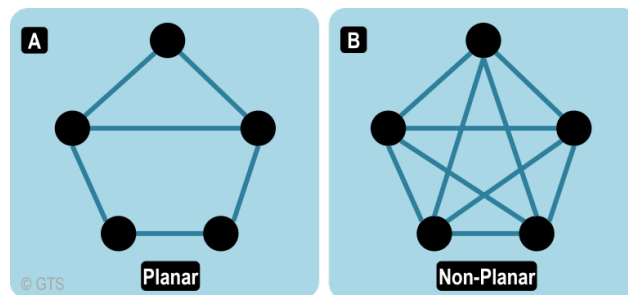
I triangoli di una mesh possono essere memorizzati come entità indipendenti, ciascuna con i propri tre vertici: questo approccio, tuttavia, comporta la duplicazione dei vertici condivisi da più triangoli, con un evidente spreco di spazio. In alternativa, è possibile memorizzare una sola volta ciascun vertice distinto e far sì che ogni triangolo vi faccia riferimento tramite un indice: questa soluzione,

nota come *indexed triangle mesh* (o *shared-vertex mesh*), evita la duplicazione dei dati di vertice ed è la forma di memorizzazione più diffusa nelle applicazioni pratiche.



2.3.3 Grafi planari e Planar Straight-Line Graph

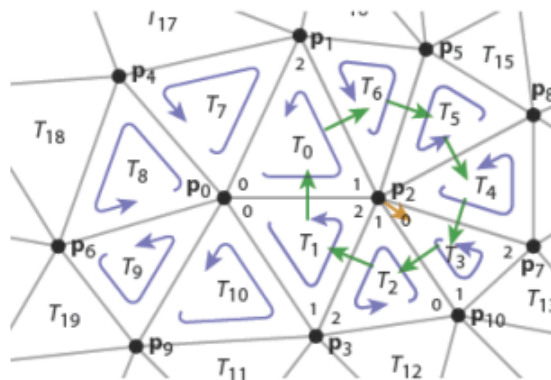
Le strutture dati per la connettività discusse nel seguito derivano concettualmente dall'idea di rappresentare una mesh come un grafo: un insieme di vertici e di archi, a cui si aggiunge un insieme di facce definite come cicli ordinati di vertici o archi. Un grafo è detto planare se può essere disegnato su un piano senza che i suoi archi si intersechino al di fuori dei vertici condivisi; è detto planare embedded (o piano) quando tale disposizione è stata effettivamente fissata nello spazio. Quando, in aggiunta, gli archi sono segmenti rettilinei anziché curve generiche, si parla di **Planar Straight-Line Graph** (PSLG): è la rappresentazione naturale per dati geometrici 2D costituiti da spezzate e poligoni, ed è tipicamente il punto di partenza per operazioni come l'estrazione delle facce racchiuse dal grafo o la sua triangolazione.



2.3.4 Strutture dati per la connettività

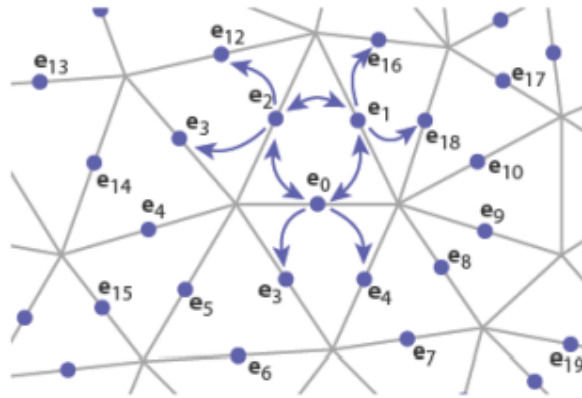
Oltre alla semplice memorizzazione indicizzata, esistono diverse strutture dati pensate per rendere esplicita ed efficiente la navigazione della connettività di una mesh, tanto più rilevanti quanto più le mesh coinvolte sono di grandi dimensioni [6].

Una prima soluzione, relativamente semplice, è la **Triangle-Neighbor Structure**: a partire da una mesh a vertici condivisi, si arricchisce ciascun triangolo con un riferimento ai tre triangoli adiacenti (uno per ciascun lato), e ciascun vertice con un riferimento a uno qualsiasi dei triangoli incidenti su di esso.

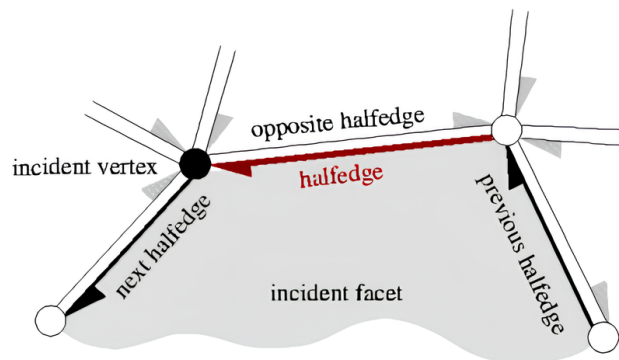


Una struttura più flessibile, ampiamente utilizzata, è la **Winged-Edge**, che memorizza l'informazione di connettività a livello di arco anziché di faccia [6]. In una mesh Winged-Edge, ogni arco memorizza un riferimento ai due vertici che collega (tipicamente indicati come vertice di testa e di coda), alle due facce di cui è parte (la faccia a sinistra e quella a destra), e, soprattutto, agli archi successivo e precedente nella percorrenza in senso antiorario di ciascuna delle due facce adiacenti. Ogni vertice e ogni faccia memorizza inoltre un riferimento a uno qualsiasi degli archi a esso incidenti. Il nome della struttura deriva dal fatto che l'insieme di questi riferimenti, disposti attorno a un singolo arco, assume graficamente la forma di una farfalla con quattro "ali".

La struttura Winged-Edge è elegante ma presenta un inconveniente pratico: per spostarsi da un arco al successivo è necessario verificare costantemente da quale lato lo si sta percorrendo, poiché un arco è condiviso da due facce e la sua orientazione dipende dal punto di vista. La struttura **Half-Edge** (o *doubly-linked edge list*) risolve questo problema suddividendo ciascun arco in due *half-edge* orientati in direzioni opposte, uno per ciascuna delle due facce che condividono l'arco: l'informazione normalmente memorizzata su un singolo arco Winged-Edge viene così distribuita



tra i due half-edge, ciascuno coerente con l'orientazione della propria faccia. Ogni half-edge memorizza un riferimento alla faccia sul proprio lato dell'arco, al vertice verso cui punta, al proprio gemello (*twin*, l'half-edge opposto che rappresenta lo stesso arco fisico visto dall'altro lato) e all'half-edge successivo lungo il perimetro della propria faccia. Percorrendo ripetutamente questi riferimenti "successivi" si ottiene un ciclo chiuso corrispondente esattamente al contorno di una faccia, mentre alternando i riferimenti al gemello e al successivo è possibile iterare sugli archi che si dipartono da un dato vertice: proprietà che rende la struttura Half-Edge particolarmente indicata per algoritmi che devono navigare ripetutamente la topologia della mesh, come l'estrazione o la classificazione delle facce racchiuse da un PSLG.



2.3.5 Attributi dei vertici

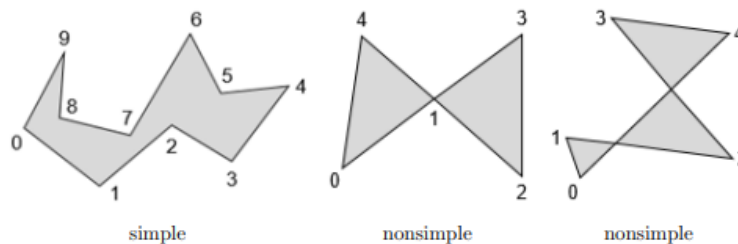
Come discusso in apertura di questa sezione, un vertice può essere arricchito con attributi opzionali oltre alla propria posizione nello spazio; tra questi, due sono di particolare rilevanza per una pipeline di rendering: il vettore normale e le coordinate texture.

Il vettore **normale** è un vettore unitario perpendicolare alla superficie nel punto considerato, ed è l'informazione che un motore di rendering utilizza per determinare l'orientazione della superficie rispetto a una sorgente luminosa ai fini del calcolo dello shading.

Le **coordinate texture** associano invece a ciascun vertice una posizione (u, v) , con $u, v \in [0, 1]$, all'interno di un'immagine bidimensionale (la texture) usata per aggiungere dettaglio visivo alla superficie senza dover infittire la geometria della mesh. Per applicare correttamente una texture a un triangolo è necessario specificare, per ciascuno dei suoi tre vertici, quale punto della texture gli corrisponde; il resto della superficie del triangolo viene poi campionato per interpolazione tra questi tre valori. Per convenzione, la coordinata $(0, 0)$ corrisponde all'angolo inferiore sinistro dell'immagine e $(1, 1)$ all'angolo superiore destro. Qualora vengano specificate coordinate al di fuori dell'intervallo $[0, 1]$, il comportamento del campionamento dipende dalla modalità di *wrapping* scelta (ripetizione, ripetizione speculare, o clamping ai bordi), un dettaglio implementativo che esula dagli scopi di questa trattazione teorica.

2.4 Algoritmi di triangolazione

Un problema classico della computer grafica è quello di scomporre un poligono semplice in un insieme di triangoli i cui vertici sono solo quelli del poligono semplice. Per definizione, un poligono semplice è una sequenza ordinata di n punti. I vertici consecutivi sono collegati da un bordo e l'ultimo bordo collega il primo e l'ultimo punto. Ogni vertice condivide esattamente due spigoli. Gli unici punti in cui è consentito l'intersezione degli spigoli sono i vertici.



La scomposizione di un poligono semplice in triangoli è chiamata triangolazione del poligono. Qualsiasi triangolazione di un poligono semplice di n vertici ha $n - 2$ triangoli.

Nel corso degli anni sono stati proposti numerosi algoritmi per risolvere il problema della triangolazione di un poligono, che si differenziano tra loro per generalità, complessità computazionale

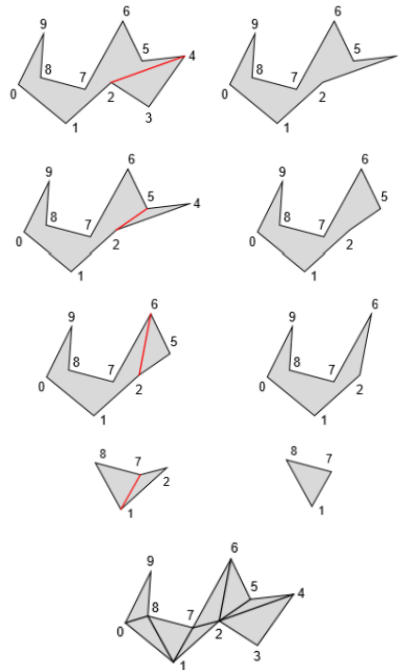
e qualità geometrica dei triangoli prodotti. In questa sezione se ne richiamano due tra i più rappresentativi: l'*ear clipping*, il più semplice, e la (*Constrained*) *Delaunay Triangulation*, in grado di produrre triangoli di qualità geometrica superiore.

2.4.1 Ear clipping

L'*ear clipping* è uno dei metodi più diretti per triangolare un poligono semplice. Il metodo si basa sulla nozione di *orecchio* (*ear*): dati tre vertici consecutivi del poligono, essi formano un orecchio se il vertice centrale è convesso e il segmento che unisce il primo e il terzo vertice è interamente contenuto all'interno del poligono, cioè se il triangolo da essi individuato può essere staccato dal poligono senza uscire dal suo contorno. Un risultato noto in geometria computazionale, il *teorema dei due orecchi*, garantisce che ogni poligono semplice con almeno quattro vertici possieda sempre almeno due orecchi.

Si individua un orecchio, lo si rimuove dal poligono (aggiungendo il triangolo corrispondente all'insieme di output), ottenendo un nuovo poligono, più piccolo, che soddisfa ancora le condizioni del teorema; l'operazione viene ripetuta fino a quando non rimane un solo triangolo.

L'algoritmo è semplice da implementare ma esegue in tempo $O(n^2)$ [3]. Esistono varianti più efficienti, come la decomposizione in poligoni monotoni ($O(n \log n)$) o algoritmi randomizzati praticamente lineari, ma sono più complesse da implementare.



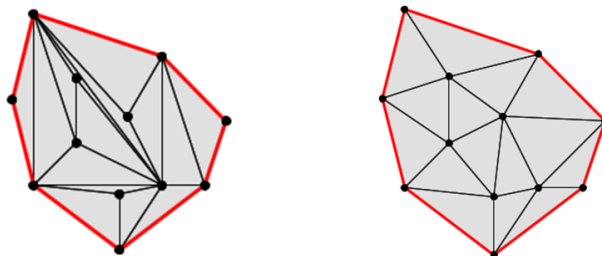
2.4.2 Triangolazione di Delaunay

Un approccio alternativo, che si applica in generale a un insieme di punti anziché a un poligono, è la triangolazione di Delaunay (*Delaunay Triangulation*, DT): dato un insieme di punti nel piano, la DT suddivide il loro inviluppo convesso in triangoli tali che la circonferenza circoscritta a ciascuno di essi (la circonferenza passante per i suoi tre vertici) non contenga, al proprio interno, nessun altro punto dell'insieme. Questa proprietà ha l'effetto di massimizzare il più piccolo angolo tra tutti i triangoli della triangolazione, evitando così la formazione di triangoli eccessivamente sottili o degeneri (*sliver*).

La DT dipende unicamente dalla posizione dei punti e non tiene conto di eventuali archi che si desidera vedere presenti nella triangolazione risultante: nulla impedisce che un triangolo la "attraversi". La Constrained Delaunay Triangulation (CDT) generalizza la DT introducendo la possibilità di imporre un insieme di segmenti obbligatori (*constraint*), che devono necessariamente comparire come archi della triangolazione finale. L'input al problema della CDT è tipicamente un PSLG: un insieme di punti e di segmenti non incidenti tra loro, questi ultimi trattati come vincoli da rispettare.

Per illustrare questa idea, la figura seguente mostra un esempio di ciò che accade quando un

insieme di punti è collegato in modo arbitrario rispetto a uno conforme al criterio di Delaunay [5]:



Dal punto di vista della complessità computazionale, sono noti diversi algoritmi in grado di calcolare la CDT di un PSLG generico in tempo $O(n \log n)$; nel caso particolare in cui l'input sia un singolo poligono semplice, la CDT può essere calcolata in tempo lineare $O(n)$.

Rispetto all'ear clipping, la CDT presenta due caratteristiche importanti: da un lato tende a produrre triangoli di qualità geometrica superiore evitando la formazione di triangoli degeneri; dall'altro consente di specificare esplicitamente quali segmenti debbano necessariamente far parte della triangolazione risultante.

3. Risultati e valutazione

4. Conclusioni

Qui andrebbero le tue conclusioni.

Ringraziamenti

Qui i ringraziamenti, se lo desideri.

Bibliografia

- [1] Autoware Foundation. *Spatial Hash*. https://autowarefoundation.github.io/autoware.universe/main/common/autoware_utils/design/spatial-hash-design/.
- [2] Jaan Tollander de Balsch. *Searching for Fixed-Radius Near Neighbors with Cell Lists Algorithm in Julia Language*. <https://jaantollander.com/post/searching-for-fixed-radius-near-neighbors-with-cell-lists-algorithm-in-julia-language/>. 2020.
- [3] David Eberly. *Triangulation by Ear Clipping*. Geometric Tools, Redmond WA 98052. Ultima modifica: 3 agosto 2024. 2002. URL: <https://www.geometrictools.com/Documentation/TriangulationByEarClipping.pdf>.
- [4] Martin Ester et al. «A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise». In: *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD-96)*. AAAI Press, 1996, pp. 226–231.
- [5] Gary W. Lucas. *An Introduction to the Delaunay Triangulation*. TinfourDocs, Tinfour Project. 2016. URL: <https://gwlucastrig.github.io/TinfourDocs/DelaunayIntro/>.
- [6] Steve Marschner e Peter Shirley. *Fundamentals of Computer Graphics*. 5^a ed. CRC Press, 2021.
- [7] scikit-learn developers. *sklearn.cluster.DBSCAN*. <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html>.
- [8] Okan Yenigün. *DBSCAN Clustering Algorithm Demystified*. <https://builtin.com/articles/dbscan>. 2024.